Lab 07 · Functions 2 and Set(함수 2와 집합)

실습 · Notion 아카이브

list comprehension(리스트 컴프리헨션)

- [표현식 for 요소 in 시퀀스자료형 [if 조건문]]

```
L = [ i ** 2 for i in range(1, 6) ]
print(L)
```

- >>> [1, 4, 9, 16, 25]

```
L1 = [1,2,3]
L2 = [3,4,5]

L = [x*y for x in L1 for y in L2]
print(L)
```

- >>> [3, 4, 5, 6, 8, 10, 9, 12, 15]
  - x 반복문 안에 y반복문이 있는 것과 같은 결과가 도출됨.

zip()

- `zip()` 은 각 리스트의 같은 인덱스 원소끼리 묶은 튜플을 원소로 하는 이터레이터를 반환한다. 리스트가 필요하면 `list(zip(list1, list2))` 처럼 `list()` 로 변환해야 한다.
- list(zip(list1, list2)) 형태로 list안에 zip을 넣어야 함

```
L = list(zip([1,2,3], [4,5,6]))
print(L)
```

- `\>\>\> [(1, 4), (2, 5), (3, 6)]`

```
L = [sum(x) for x in zip((1,2,3),(4,5,6),(7,8,9))]
print(L)
```

- >>> [12, 15, 18]

실습 1

```
'아래와 같이 주어진 2개의 리스트로 딕셔너리를 만들어 출력'
L1 = ['one','two','three','four']
L2 = [1,2,3,4]
```

```
result = dict(zip(L1,L2))
print(result)
```

가변Parameter(매개변수)

- **Function(함수)** ⇒ *매개변수

```
'짝수 개수 구하기'


def count_even(*n):
    cnt = 0
    for v in n:
        if v % 2 == 0:
            cnt += 1
    return cnt


print(count_even(1,2,3,4,5,6,7,8,9))
```

실습 2

```
'1개 이상의 3차원 벡터들을 전달받아 벡터들의 합을 구하여 반환'


def sum_vectors(*vectors):
    return [sum(vals) for vals in zip(*vectors)]


v1 = [0,1,2]
v2 = [0.5, 0.5, 0.5]
v3 = [1,3,5]


vector_sum = sum_vectors(v1,v2,v3)
print(vector_sum)
```

디폴트 매개변수

- 매개변수의 기본값을 지정

```
def print_name(first, second='Kim'):
    print(f'My name is {first} {second}.')


#함수 호출
print_name('HoChang')
```

- >>> My name is HoChang Kim.

키워드 매개변수

- 함수 호출 시, 해당 매개변수 이름을 명시적으로 지정해서 전달
- 즉, 호출할 때 순서보단 x=10, y=20 처럼 키워드로 입력

```
def calc(x,y=0,z=0):
    return x+y+z


result = calc(y=20, x=10, z=30)
print(result)
```

- >>> 60
- calc(10,30,y=20) 이렇게 이미 y자리 입력하고 키워드에 넣으면 오류 발생

실습 3

```
'숫자로 구성된 리스트 2개'
'=> 리스트 합친 후 정렬한 새로운 리스트 반환'


def merge_list(l1=[0],l2=[0]):
    ml = l1 + l2
    ml.sort()
    return ml



if __name__ == '__main__':
    l = [3, 5, 9, 1, 2]
    ml1 = merge_list(l,[2,1])
    ml2 = merge_list([6,9,4])
    ml3 = merge_list()
    print(ml1) # [1, 1, 2, 2, 3, 5, 9]
    print(ml2) # [0, 4, 6, 9]
    print(ml3) # [0, 0]
```

set(집합)

- 순서 없이 항목 저장, 항목이 중복될 수 없음

```
s1 = set([1,2,3,3,3,4,2])
print(s1)
```

- >>> \{1,2,3,4\}

- set() 내장 함수

```
s = set([1,2,3,3,3,4,2])
print(s)
{1, 2, 3, 4}

#add
s.add(10) #=> 단일 요소를 더함
print(s)
{1, 2, 3, 4, 10}

#remove
s.remove(2)
print(s)
{1, 3, 4, 10}

#update
s.update([1,3,5,7,9]) #=> 리스트를 더함
print(s)
{1, 3, 4, 5, 7, 9, 10}

#discard
s.discard(7) #=> 7을 제거
print(s)
{1, 3, 4, 5, 9, 10}

#clear
s.clear()
print(s)
set()
```

- 집합 관련 내장 함수

```
s1 = {1,2,3}
s2 = {3,4,5}

#합집합(변경x,반환)
print(s1.union(s2))
print(s1|s2)
{1, 2, 3, 4, 5}
```

```
#교집합(변경x,반환)
print(s1.intersection(s2))
print(s1&s2)
{3}

#차집합(변경x,반환)
print(s1.difference(s2))
print(s1 - s2)
{1, 2}
```

실습 4

```
'두 사람이 등산한 산의 목록이 각각 집합(set)으로 주어질 때, 두 사람이'
'모두 방문한 산과 한쪽만 방문한 산을 구하여 반환'

def compare_mountains(h1,h2):
    both = h1&h2
    one = (h1-h2)|(h2-h1)
    return both, one

if __name__ == '__main__':
    hiker_1 = {'설악산', '한라산', '지리산', '태백산'}
    hiker_2 = {'설악산', '주왕산', '덕유산', '한라산'}
    hiker_3 = {'무장산', '주왕산'}
    both1, one1 = compare_mountains(hiker_1, hiker_2)
    both2, one2 = compare_mountains(hiker_1, hiker_3)
    print(both1, one1)
    "{'한라산', '설악산'} {'지리산', '태백산', '덕유산', '주왕산'}"
    print(both2, one2)
    "set() {'한라산', '지리산', '태백산', '무장산', '설악산', '주왕산'}"
```

← 프로그래밍과 문제해결 · CSED101 목차로